



CENTRO UNIVERSITÁRIO EUROAMERICANO – UNIEURO

Curso de Análise e Desenvolvimento de Sistemas e Sistemas de Informação

PROJETO DE PESQUISA SIMBIONTE

Simulação de Ecossistema de Influência Digital

Configuração do Ambiente, Autenticação e Validação de Segurança

Subgrupo D – Segurança e Regras

Pesquisadores Responsáveis: Ana Luiza, Vitor e Yasmin

Brasília - DF

2026

1. Introdução e Objetivo

Este documento apresenta as atividades desenvolvidas pelo **Subgrupo D – Segurança e Regras** durante a Etapa 3 da Iniciação Científica **Projeto Simbionte – Simulação de Ecossistema de Influência Digital**.

O objetivo deste trabalho foi elaborar a documentação técnica referente ao fluxo de autenticação da aplicação, definir o contrato da API utilizado nesta etapa do projeto, estabelecer critérios para validação dos mecanismos de segurança e construir uma matriz de testes voltada aos principais cenários de exceção relacionados à autenticação.

Como parte das atividades de pesquisa, o Subgrupo D também desenvolveu um módulo independente de criptografia utilizando a biblioteca **bcrypt**, com o propósito de demonstrar a aplicação das boas práticas de armazenamento e verificação de credenciais em aplicações web.

Além da produção da documentação técnica, foi realizada uma revisão da implementação disponibilizada pelo Subgrupo A, responsável pelo desenvolvimento do back-end da aplicação. A análise teve como finalidade verificar a conformidade da implementação com os requisitos de segurança definidos para esta etapa da pesquisa, contemplando aspectos relacionados ao armazenamento seguro de senhas, autenticação baseada em JSON Web Tokens (JWT), proteção das rotas privadas, gerenciamento de segredos, renovação de sessão e tratamento de erros.

2. Contratos da API e Arquitetura do Fluxo de Autenticação

O processo de autenticação adotado na aplicação segue práticas consolidadas de segurança para proteção das credenciais dos usuários e controle de acesso aos recursos protegidos da API.

A autenticação utiliza **JSON Web Tokens (JWT)** para identificação dos usuários autenticados. O armazenamento das senhas é realizado por meio da biblioteca **bcrypt**, responsável pela geração de hashes unidirecionais utilizando salt automático, aumentando a resistência contra ataques de força bruta e tabelas rainbow.

Na implementação analisada, o custo de processamento utilizado pelo **bcrypt** é definido diretamente na aplicação por meio da constante **BCRYPT_COST = 12**, enquanto a validação das credenciais é realizada utilizando a função **bcrypt.compare()**.

Todas as requisições e respostas da API utilizam o formato **application/json**, e os códigos de resposta seguem o padrão definido pelo protocolo HTTP.

2.1. Endpoints Públicos

POST /auth/register

Endpoint responsável pelo cadastro de novos usuários.

Payload obrigatório

- nome (string)
- email (string válida e única)
- senha (string)

Resposta — 201 Created

```
{  
  "usuario": {  
    "id": "...",  
    "nome": "...",  
    "email": "..."  
  }  
}
```

Resposta — 409 Conflict

```
{  
  "error": {  
    "code": "...",  
    "message": "..."  
  }  
}
```

Retornada quando o e-mail informado já está cadastrado.

POST /auth/login

Endpoint responsável pela autenticação dos usuários cadastrados.

Payload obrigatório

- email (string)
- senha (string)

Resposta — 200 OK

```
{  
  "usuario": {  
    "id": "...",  
    "nome": "...",  
    "email": "..."  
  },  
  "accessToken": "...",  
  "refreshToken": "..."  
}
```

O Access Token deve ser utilizado para acessar as rotas protegidas da API.

O Refresh Token é utilizado para renovação da sessão do usuário.

Resposta — 401 Unauthorized

```
{  
  "error": {  
    "code": "...",  
    "message": "..."  
  }  
}
```

Retornada quando as credenciais informadas são inválidas.

2.2. Endpoints Privados

As rotas protegidas exigem que o cliente envie um Access Token JWT no cabeçalho HTTP utilizando o formato:

Authorization: Bearer <access_token>

Durante esta etapa do projeto foram implementados os seguintes endpoints protegidos:

GET /me

Retorna os dados do usuário autenticado após validação do Access Token.

POST /auth/refresh

Responsável pela renovação da sessão.

Recebe um Refresh Token válido e retorna um novo par de tokens (Access Token e Refresh Token), invalidando automaticamente o token anteriormente utilizado, conforme a estratégia de rotação implementada.

POST /auth/logout

Responsável por invalidar o Refresh Token armazenado, encerrando a sessão do usuário autenticado.

Caso o Access Token esteja ausente, inválido ou expirado, a API retorna:

```
{
  "error": {
    "code": "...",
    "message": "..."
  }
}
```

Na implementação analisada, todas as falhas relacionadas à autenticação (token ausente, inválido, adulterado ou expirado) retornam o código HTTP **401 Unauthorized**.

As funcionalidades relacionadas à criação de agentes, publicações, comentários, dashboard, TrustScore e demais recursos do ecossistema Simbionte ainda não fazem parte da

implementação analisada nesta etapa da pesquisa e, portanto, não possuem endpoints implementados na versão avaliada.

3. Artefato Desenvolvido

Além da documentação técnica e das atividades de validação, o Subgrupo D desenvolveu um módulo independente de criptografia utilizando a biblioteca **bcrypt**, com o objetivo de demonstrar a aplicação prática dos mecanismos de proteção de credenciais adotados pelo Projeto Simbionte.

O módulo implementa funções responsáveis pela geração de hashes de senha e pela verificação segura das credenciais informadas durante o processo de autenticação. Para isso, foram utilizadas as bibliotecas **bcrypt** e **dotenv**, permitindo a organização da lógica de segurança em um componente reutilizável e de fácil integração.

A implementação contempla:

- geração de hashes de senha utilizando bcrypt;
- verificação de credenciais por meio da função `bcrypt.compare()`;
- tratamento de exceções durante os processos de geração e validação dos hashes;
- configuração do ambiente de execução por meio de arquivo `.env`;
- rotina de testes para validação do funcionamento do módulo.

O código-fonte foi disponibilizado no repositório do Subgrupo D (https://github.com/vitorguino/Simbionte_Security.git) como evidência das atividades desenvolvidas durante esta etapa da pesquisa.

Durante a revisão da implementação disponibilizada pelo Subgrupo A verificou-se que a aplicação já possuía mecanismo equivalente para geração e validação de hashes de senha integrado ao serviço de autenticação. Dessa forma, o módulo desenvolvido pelo Subgrupo D permaneceu como artefato técnico da pesquisa e como evidência da implementação realizada pelo grupo, sem substituir a implementação adotada no projeto integrado.

4. Validação da Implementação de Segurança

Após a elaboração da documentação técnica e do módulo de criptografia, foi realizada uma revisão da implementação disponibilizada pelo Subgrupo A com o objetivo de verificar a

conformidade entre os requisitos de segurança definidos para esta etapa da pesquisa e os mecanismos efetivamente implementados na aplicação.

A validação foi conduzida por meio da análise do código-fonte, da documentação disponibilizada pelo Subgrupo A e da especificação da API apresentada na documentação OpenAPI (Swagger).

Durante a revisão técnica foram analisados os seguintes mecanismos:

Criptografia das Credenciais

Verificou-se que a aplicação utiliza a biblioteca **bcrypt** para geração do hash das senhas durante o cadastro dos usuários. No processo de autenticação, a comparação entre a senha informada e o hash armazenado é realizada utilizando a função **bcrypt.compare()**, conforme recomendado pelas boas práticas de segurança para aplicações web.

Resultado da validação: Conforme à implementação analisada.

Autenticação Baseada em JWT

A aplicação utiliza **JSON Web Tokens (JWT)** para autenticação dos usuários.

Os Access Tokens são assinados utilizando segredo armazenado em variáveis de ambiente e possuem tempo de expiração configurável. Após a autenticação, o Access Token é utilizado para acesso às rotas protegidas da aplicação.

Resultado da validação: Conforme à implementação analisada.

Proteção das Rotas Privadas

O middleware **authGuard** realiza a validação das requisições destinadas às rotas protegidas.

Durante a análise verificou-se que o middleware:

- verifica a existência do cabeçalho Authorization;
- valida o formato Bearer Token;
- verifica a assinatura do JWT;
- impede o acesso quando o token está ausente, inválido ou expirado;

- disponibiliza os dados do usuário autenticado para as rotas protegidas.

Na implementação analisada, todas as falhas de autenticação relacionadas ao Access Token retornam o código HTTP **401 Unauthorized**.

Resultado da validação: Conforme à implementação analisada.

Gerenciamento Seguro de Chaves

Os segredos utilizados para assinatura dos JWTs e demais configurações sensíveis são armazenados em variáveis de ambiente e carregados durante a inicialização da aplicação.

Essa abordagem evita a exposição de informações sensíveis diretamente no código-fonte.

Resultado da validação: Conforme à implementação analisada.

Refresh Token

A implementação utiliza Refresh Tokens gerados por meio de valores aleatórios criptograficamente seguros.

Os tokens são armazenados no Redis após aplicação de hash SHA-256 e seguem estratégia de rotação, invalidando automaticamente o token anterior após sua utilização para renovação da sessão.

Resultado da validação: Conforme à implementação analisada.

Proteção Contra Ataques

Durante a revisão identificou-se a utilização das bibliotecas **express-rate-limit** para limitação de tentativas de autenticação e **helmet** para configuração automática dos principais cabeçalhos HTTP relacionados à segurança da aplicação.

O mecanismo de rate limiting é aplicado às rotas de cadastro e autenticação, reduzindo riscos relacionados a ataques de força bruta.

Resultado da validação: Conforme à implementação analisada.

Tratamento de Erros

A aplicação diferencia erros de validação, autenticação e erros internos do servidor por meio de respostas padronizadas.

Durante a revisão verificou-se que as respostas seguem estrutura consistente baseada em objetos contendo código e mensagem de erro.

Resultado da validação: Conforme à implementação analisada.

5. Execução da Matriz de Qualidade (QA) - Testes de Caminho Triste

5.1 Metodologia

A validação experimental foi realizada por meio da execução de uma coleção de testes desenvolvida no Postman, construída a partir dos cenários definidos pelo Subgrupo D e da implementação disponibilizada pelo Subgrupo A.

Os testes contemplaram os principais cenários relacionados ao cadastro de usuários, autenticação, gerenciamento de sessão, proteção das rotas privadas e tratamento de exceções.

A coleção utilizou variáveis de ambiente como:

- {{base_url}}
- {{email_valido}}
- {{senha_invalida}}
- {{access_token}}
- {{refresh_token}}

5.2 Matriz de Testes

ID	Vetor de Teste (Módulo)	Condição Simulada	Retorno Observado	Status de Validação
QA-01	Cadastro (Validação)	Submissão com omissão de campos ou senha inferior a 8 caracteres.	400 Bad Request	Aprovado

ID	Vetor de Teste (Módulo)	Condição Simulada	Retorno Observado	Status de Validação
QA-02	Cadastro (Unicidade)	Tentativa de inserção de e-mail preexistente no banco de dados.	409 Conflict	Aprovado
QA-03	Login (Inexistente)	Fornecimento de e-mail não catalogado na persistência.	401 Unauthorized	Aprovado
QA-04	Login (Falha Senha)	E-mail válido com senha cuja verificação bcrypt falha.	401 Unauthorized	Aprovado
QA-05	Acesso (Ausência JWT)	Requisição a endpoint privado (GET /me) sem cabeçalho Authorization.	401 Unauthorized	Aprovado
QA-06	Acesso (Adulteração)	Submissão de token manipulado e/ou expirado.	401 Unauthorized	Aprovado
QA-07 a QA-09	IA / Engajamento	Injeções de Prompt e abusos de taxa em agentes e posts.	N/A (Rotas aguardando implementação futura)	Adiado (Escopo Futuro)
QA-10	Vulnerabilidade <i>Input</i>	Injeção de <i>scripts</i> maliciosos em campos de texto plano.	400 Bad Request	Aprovado
QA-11	Brute Force	Ataque de força bruta por múltiplas requisições sucessivas (Limiar de 20 req/15min).	429 Too Many Requests	Aprovado

5.3 Resultados Obtidos

Os resultados obtidos demonstram que a implementação analisada apresentou comportamento compatível com os requisitos de segurança previstos para a etapa de autenticação do Projeto Simbionte.

Durante a execução dos testes foram validados mecanismos relacionados ao cadastro de usuários, autenticação baseada em JSON Web Tokens (JWT), utilização de Refresh Tokens, proteção das rotas privadas, limitação de tentativas de autenticação e tratamento das respostas de erro da API.

A comparação entre os resultados esperados e os resultados observados permitiu verificar a conformidade da implementação disponibilizada pelo Subgrupo A com os requisitos definidos pelo Subgrupo D para esta etapa da pesquisa.

6. Conclusão da Validação

As atividades desenvolvidas pelo Subgrupo D contemplaram a elaboração da documentação técnica do fluxo de autenticação, a revisão da implementação disponibilizada pelo Subgrupo A, o desenvolvimento de um módulo independente de criptografia utilizando bcrypt e a execução da validação experimental por meio de uma coleção de testes desenvolvida no Postman.

Durante a análise foram confirmados mecanismos como armazenamento seguro de credenciais utilizando bcrypt, autenticação baseada em JSON Web Tokens (JWT), proteção das rotas privadas por meio do middleware authGuard, utilização de Refresh Tokens, gerenciamento de informações sensíveis por variáveis de ambiente, limitação de requisições com express-rate-limit e utilização da biblioteca helmet.

A execução da matriz de testes demonstrou conformidade entre os resultados esperados e o comportamento observado na aplicação, corroborando a revisão técnica realizada pelo Subgrupo D e evidenciando que a implementação atende aos requisitos de segurança definidos para a Etapa 3 do Projeto Simbionte dentro do escopo atualmente desenvolvido.

O módulo de criptografia desenvolvido pelo Subgrupo D constituiu um artefato técnico da pesquisa, demonstrando a aplicação prática dos conceitos estudados durante a Iniciação Científica e complementando as atividades de documentação e validação realizadas nesta etapa do projeto.

7. Apêndice A - Matriz de Automação de QA (Postman Collection Atualizada)

Para fins de auditoria e reprodutibilidade, o arquivo JSON abaixo contém a coleção de requisições estruturada no Postman, atualizada para refletir as rotas oficiais (/auth/register e

/auth/login) e as exigências reais da aplicação, servindo como comprovação técnica da execução de testes do Subgrupo D.

```
{
  "info": {
    "name": "QA Simbionte - Subgrupo D (Implementação Real)",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "QA-01: Cadastro - Senha Curta (Zod Validation)",
      "request": {
        "method": "POST",
        "header": [{"key": "Content-Type", "value": "application/json"}],
        "url": "{{base_url}}/auth/register",
        "body": {
          "mode": "raw",
          "raw": "{\n  \"nome\": \"Teste QA\",\n  \"email\": \"{{email_valido}}\",\n  \"senha\": \"123\\n\"}"
        }
      }
    },
    {
      "name": "QA-02: Cadastro - Email Conflict",
      "request": {
        "method": "POST",
        "header": [{"key": "Content-Type", "value": "application/json"}],
        "url": "{{base_url}}/auth/register",
        "body": {
          "mode": "raw",
          "raw": "{\n  \"nome\": \"Teste QA\",\n  \"email\": \"{{email_valido}}\",\n  \"senha\": \"{{senha_valida_min8}}\\n\""
        }
      }
    },
    {
      "name": "QA-03 e QA-04: Login - Falha de Credenciais",
      "request": {
        "method": "POST",
        "header": [{"key": "Content-Type", "value": "application/json"}],
        "url": "{{base_url}}/auth/login",
        "body": {
          "mode": "raw",
          "raw": "{\n  \"email\": \"{{email_valido}}\",\n  \"senha\": \"{{senha_invalida}}\\n\""
        }
      }
    },
    {
      "name": "QA-05 e QA-06: Rota Privada - Falhas de JWT",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token_invalido_ou_expirado}}"}],
        "url": "{{base_url}}/me"
      }
    }
  ]
}
```

```

"name": "QA-10: Cadastro - Injeção de Script/SQL (Input Validation)",
  "request": {
    "method": "POST",
    "header": [{"key": "Content-Type", "value": "application/json"}],
    "url": "{{base_url}}/auth/register",
    "body": {
      "mode": "raw",
      "raw": "{\n  \"nome\": \"\", \n  \"email\": \"admin' OR 1=1 --@teste.com\", \n  \"senha\": \"senha123\"\n}"
    }
  },
  {
    "name": "QA-11: Brute Force (Rate Limit 20/15min)",
    "request": {
      "method": "POST",
      "header": [{"key": "Content-Type", "value": "application/json"}],
      "url": "{{base_url}}/auth/login",
      "body": {
        "mode": "raw",
        "raw": "{\n  \"email\": \"{{email_valido}}\", \n  \"senha\": \"{{senha_invalida}}\"\n}"
      }
    }
  }
]
}

```

Referências

- OWASP FOUNDATION. **OWASP API Security Top 10**. Disponível em: <https://owasp.org/www-project-api-security/>
- IETF. **RFC 7519 - JSON Web Token (JWT)**. Disponível em: <https://datatracker.ietf.org/doc/html/rfc7519>
- PROVOS, Niels; MAZIÈRES, David. *A Future-Adaptable Password Scheme*. Disponível em: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>